ЛАБОРАТОРНАЯ РАБОТА № 2

Тема: **ГРАФИКА В WINDOWS FORMS**

1 ЦЕЛЬ РАБОТЫ

Познакомиться:

- с программными средствами C#, позволяющими выводить данные на экран;
- с основными возможностями технологии GDI+;
- с классами, входящими в GDI+.

Научиться:

- создавать объекты Graphics для рисования;
- рисовать фигуры и изображения на форме.

2 ОБЪЁМ ЛАБОРАТОРНОЙ РАБОТЫ И ПОРЯДОК ЕЁ ВЫПОЛНЕНИЯ

Данная лабораторная работа выполняется в течение двух часов. Выполнение работы идет одновременно с изучением теоретической части. Для закрепления материала необходимо выполнить упражнения для самостоятельной работы.

3 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ. ТЕХНОЛОГИЯ ВЫПОЛНЕНИЯ РАБОТЫ

3.1 Общие сведения о графике Windows Forms

3.1.1 Общие сведения о GDI+

GDI+ – это набор программных средств, которые используются в .NET.

GDI+ позволяют создателям приложений выводить данные на экран или на принтер без необходимости обеспечивать работу с определенными типами устройств отображения. Для отображения информации программисту достаточно вызывать нужные методы классов GDI+. При этом автоматически учитываются типы определенных устройств и выполняются вызовы соответствующих драйверов.

Доступ к API-интерфейсу GDI+ осуществляется через набор классов, из которых создается управляемый код. Этот набор классов называется интерфейсом управляемых классов GDI+. Интерфейс управляемых классов состоит из следующих пространств имен:

System.Drawing;
System.Drawing.Drawing2D;
System.Drawing.Imaging;
System.Drawing.Text;
System.Drawing.Printing.

Достаточно обращаться к методам, содержащимся в классах GDI+. Эти методы, в свою очередь, осуществляют вызовы драйверов определенных устройств. Таким образом, GDI+ изолирует приложение от оборудования, работающего с графикой. Именно такая изоляция дает возможность создавать приложения, не зависящие от устройств.

3.1.2 Создание объектов Graphics для рисования

Перед тем как рисовать линии и фигуры, отображать текст, выводить изображения и управлять ими в GDI+ необходимо создать объект Graphics. Объект Graphics представляет поверхность рисования GDI+ и используется для создания графических изображений.

Этапы работы с графикой:

1. Создание объекта Graphics.
2. Использование объекта Graphics для рисования линий и фигур, отображения текста или изображения и управления ими.

Существуют различные способы создания графических объектов.

Способ 1. Получите ссылку на объект Graphics через объект PaintEventArgs при обработке события Paint формы или элемента управления. Это обычный способ получения ссылки на графический объект при создании кода рисования элементов управления. Подобным образом можно получить графический объект как свойство объекта PrintPageEventArgs при обработке события PrintPage для PrintDocument.

Получение ссылки на объект Graphics из объекта PaintEventArgs в событии Paint:

– объявите объект Graphics;
– присвойте переменной ссылку на объект Graphics, передаваемый как часть PaintEventArgs;
– вставьте код для рисования формы или элемента управления.

В следующем примере показано, как создавать ссылку на объект Graphics из объекта PaintEventArgs события Paint.

```
private void Form1_Paint(object sender, PaintEventArgs e)
    {
      // Объявляем объект Graphics и присваиваем ему ссылку
      // на свойство Graphics объекта PaintEventArgs.
      Graphics g = e.Graphics;

      // Вставить здесь код для рисования на форме.
    }
```

Способ 2. Вызовите метод CreateGraphics элемента управления или формы, чтобы получить ссылку на объект Graphics, соответствующий по-

верхности рисования этой формы или элемента управления. Используйте этот подход, если необходимо рисовать на поверхности уже существующей формы или элемента управления.

```
Graphics g;
/* Устанавливаем g графическим объектом, представляющим собой
поверхность для рисования элемента управления или формы,
чьим членом он является*/
g = this.CreateGraphics();
/* Вставить здесь код для рисования на элементе управления
или форме*/
```

Способ 3. Создайте объект Graphics из любого объекта, унаследованного от класса Image. Этот способ используется, когда требуется изменить существующее изображение.

```
Bitmap myBitmap = new Bitmap(@"C:\Documents and Settings\Joe\Pics\myPic.bmp");
Graphics g = Graphics.FromImage(myBitmap);
```

3.1.3 Рисование фигур и изображений и управление ими

После создания объекта Graphics его можно использовать для рисования линий и фигур, отображения текста или изображения и управления ими. Ниже представлены основные объекты, используемые с объектом Graphics.

– класс Pen – служит для рисования линий, контуров и отрисовки других геометрических объектов;

– класс Brush – служит для заливки областей, например фигур, изображений или текста;

– класс Font – содержит описание фигур, которые должны использоваться при отображении текста;

– структура Color – содержит различные цвета.

Объекты Color, Brush и Pen формируют облик графического элемента. Объект Brush применяют для отображения закрашенных фигур, Pen – для отображения прямых и дуг, а объект Color задает цвет фигуры.

Структура Color располагается в пространстве имен System.Drawing и представляет некоторый цвет. Каждый цвет определяется четырьмя значениями: Alpha, представляющим прозрачность, и тремя значениям Red, Green и Blue из диапазона 0-255. Если вы не используете эффект прозрачности, параметр Alpha можно пропустить, указав только значения Red, Green и Blue. Кроме того, множество стандартных цветов доступно в .NET Framework в виде именованных цветов, например: Tomato, Red, Yellow и др.

Объект Brush представляет кисть и служит для рисования закрашенных фигур. Все разновидности кистей являются потомками абстрактного класса Brush и позволяют создавать закрашенные фигуры различного вида. Типы

кистей и пространства имен, в которых расположены соответствующие объекты, описаны в таблице 1.

Таблица 1 – Типы кистей

Тип	Пространство имен	Описание
SolidBrush	System. Drawing	Сплошная одноцветная кисть
TextureBrush	System. Drawing	Кисть для заливки замкнутого контура изображением
HatchBrush	System. Drawing. Drawing2D	Кисть для узорной заливки
Linear Gradient Brush	System. Drawing. Draw ing2D	Кисть для заливки двухцветным градиентом
Path Gradient Brush	System.Drawing. Drawing2D	Кисть, использующая сложную градиентную запивку

Для создания объекта SolidBrush достаточно указать цвет, например:

SolidBrush myBrush = new SolidBrush(Color.PapayaWhip).

Конструкторы других кистей сложнее и требуют дополнительных параметров.

Объекты Pen представляют перья, они предназначены для рисования прямых и дуг, а также для применения к контурным фигурам различных графических эффектов. Существует только один класс Pen, и он является запечатанным (sealed).

Создать объект класса Pen несложно, достаточно указать его цвет: Pen

myPen = new Pen(Color.BlanchedAlmond).

В результате исполнения предыдущего кода создается перо, ширина которого по умолчанию равна 1 пикселю. Ширину задают в конструкторе объекта Pen следующим образом:

Pen myPen = new Pen(Color.Lirne, 4).

Этот код назначает ширину пера, равную четырем пикселам. Перо можно создать и на основе имеющейся кисти, вид таких перьев будет согласован со стилем интерфейса. Это особенно удобно при использовании сложных теней и других эффектов. Следующий пример демонстрирует создание пера на основе имеющейся кисти myBrush:

Pen myPen = new Pen(myBrush).

При создании пера на основе кисти также разрешается задавать ее ширину.

Упражнение 1

В этом упражнении вы научитесь рисовать линию на форме.

Обычно при рисовании на форме обрабатывается событие Paint формы и выполняется рисование с помощью свойства Graphics класса PaintEven- tArgs.

1. Создайте проект **Form2_1**.
2. Измените размер формы. Для этого, в окне Свойства измените значения свойства Size: ширина – 600, длина – 300.

3. В окне Свойства переключитесь на вкладку работы с событиями элементов управления, нажав на кнопку События , и дважды кликните по событию Paint – справа появится надпись Form1_Paint. В результате в программный код будет добавлен метод-обработчик события рисования на форме с тем же именем (рисунок 1).

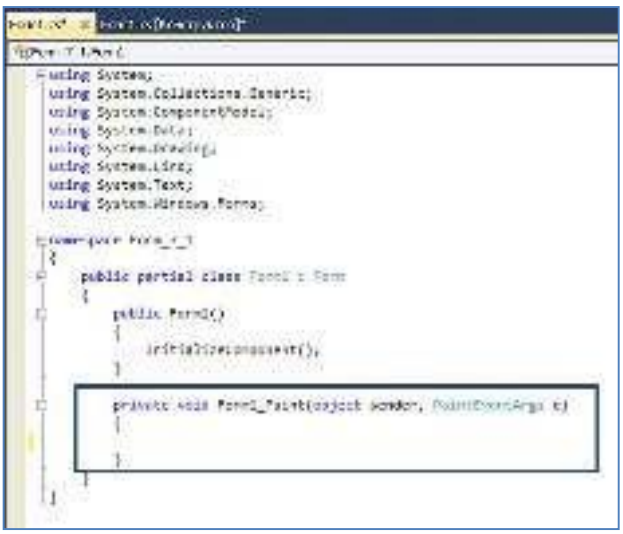

Рисунок 1 – Добавление метода-обработчика события рисования на форме в программный код

4. Добавьте в проект следующий код

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
        //Рисуем линию максимальной непрозрачности,
        //красного цвета и толщиной 6 пикселей
        Pen pen = new Pen(Color.FromArgb(255, 255, 0, 0),6);

        //Координаты линии: (20, 10) - начало линии, (300, 100) - конец линии
        e.Graphics.DrawLine(pen, 20, 10, 300, 100);
}
```

5. Запустите программу на выполнение.
6. Сравните результат работы программы с рисунком 2.
7. Сохраните проект и закройте его.

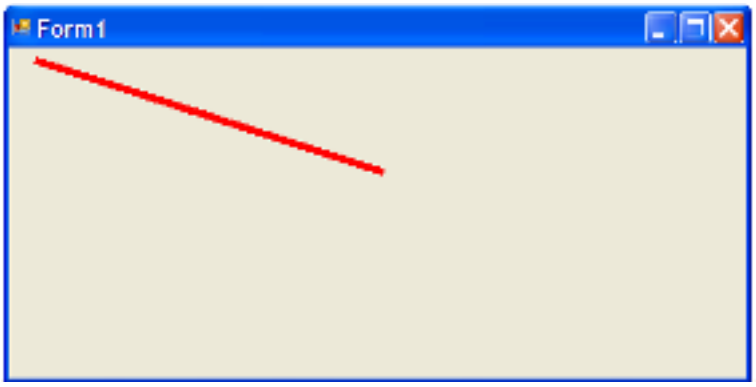

Рисунок 2 – Результат выполнения программы

Упражнение 2

В этом упражнении вы научитесь рисовать эллипс и прямоугольник на форме.

1. Создайте проект **Form2_2**.
2. Измените размер формы. Для этого, в окне Свойства измените значения свойства Size: ширина – 400, длина – 500.
3. Измените цвет фона формы на белый с помощью свойства BackColor.
4. В окне Свойства переключитесь на вкладку работы с событиями элементов управления, нажав на кнопку События , и дважды кликните по событию Paint – справа появится надпись Form1_Paint. В результате в программный код будет добавлен метод-обработчик события рисования на форме с тем же именем.
5. Добавьте в проект следующий код

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace Form2_2
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }
      private void Form1_Paint(object sender, PaintEventArgs e)
      {
        //Метод для рисования эллипса
        DrawEllipse();
```

```
            //Метод для рисования прямоугольника
            DrawRectangle();
        }
        //Рисуем эллипс

        private void DrawEllipse()
        {
            //Рисуем контур зеленого цвета, толщина 4 пикселя
            Pen myPen = new Pen(Color.Green, 4);

            Graphics formGraphics;

            formGraphics = this.CreateGraphics();

            //Задаем координаты эллипса и его размеры
            formGraphics.DrawEllipse(myPen, new Rectangle(50, 50, 200, 300));

            myPen.Dispose();

            formGraphics.Dispose();
        }

        //Рисуем прямоугольник
        private void DrawRectangle()

        {
            //Рисуем контур цвета Tomato, толщина 3 пикселя
            Pen myPen = new Pen(Color.Tomato, 3);

            Graphics formGraphics;

            formGraphics = this.CreateGraphics();

            //Задаем координаты прямоугольника и его размеры (ширина, длина)
            formGraphics.DrawRectangle(myPen, new Rectangle(70, 70, 200, 300));

            myPen.Dispose();

            formGraphics.Dispose();
        }
    }
}
```

6. Запустите программу на выполнение. Сравните результат выполнения программы с рисунком 3.
7. Сохраните проект и закройте его.

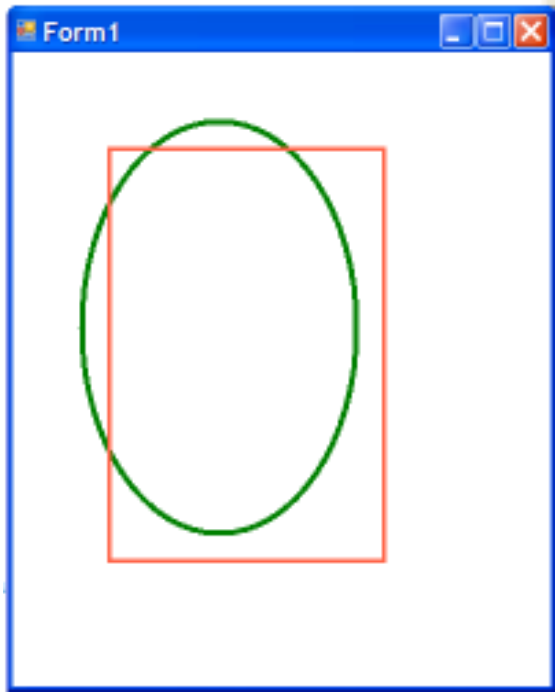

Рисунок 3 – Результат выполнения программы

Упражнение 3

В этом упражнении вы научитесь выполнять заливку указанной области.

1. Откройте проект **Form2_2**.
2. Отредактируйте код программы в соответствии с приведенным ниже кодом

```
//Рисуем эллипс
        private void DrawEllipse()
        {
          //Рисуем контур зеленого цвета, толщина 4 пикселя
          Pen myPen = new Pen(Color.Green, 4);

          Graphics formGraphics;

          formGraphics = this.CreateGraphics();

          //Задаем координаты эллипса и его размеры
          formGraphics.DrawEllipse(myPen, new Rectangle(50, 50, 200, 300));

          //Выполняем заливку эллипса желтым цветом
          formGraphics.FillEllipse(Brushes.Yellow, 50, 50, 200, 300);

          myPen.Dispose();

          formGraphics.Dispose();
         }
```

3. Запустите программу на выполнение.
4. Добавьте программный код, позволяющий выполнить заливку прямо-

угольника сиреневым цветом (таблица 2).

5. Сохраните проект и закройте его.

Таблица 2 – Таблица цветов

Цвет в C#	Название
Black	чѐрный
Maroon	бордовый
Green	зелѐный
Olive	оливковый
Navy	темно-синий
Purple	пурпурный
Teal	морской воды
Gray	серый
Silver	светло-серый
Red	красный
Lime	лимонный
Blue	синий
Yellow	жѐлтый
Fuchsia	сиреневый
Aqua	голубой
White	белый

Упражнение 4

В этом упражнении вы научитесь выполнять отрисовку текста на форме.

1. Откройте проект **Form2_2**.
2. Отредактируйте код программы в соответствии с приведенным ниже кодом

```
private void Form1_Paint(object sender, PaintEventArgs e)
    {
      //Метод для рисования эллипса
      DrawEllipse();

      //Метод для рисования прямоугольника
      DrawRectangle();

      //Метод для рисования текстовой строки
      DrawString();
    }

//Прорисовываем текст "РИСУНОК" на форме

    public void DrawString()
    {
      Graphics formGraphics = this.CreateGraphics();

      string drawString = "РИСУНОК";
```

```
        Font drawFont = new Font("Arial", 18);

        SolidBrush drawBrush = new

        System.Drawing.SolidBrush(Color.Blue);

        float x = 150.0F;

        float y = 20.0F;

        StringFormat drawFormat = new StringFormat();

        formGraphics.DrawString(drawString, drawFont, drawBrush, x, y,
        drawFormat);

        drawFont.Dispose();

        drawBrush.Dispose();

        formGraphics.Dispose();
    }
```

3. Запустите программу на выполнение и сравните результат с рисунком 4.
4. Сохраните проект и закройте его.

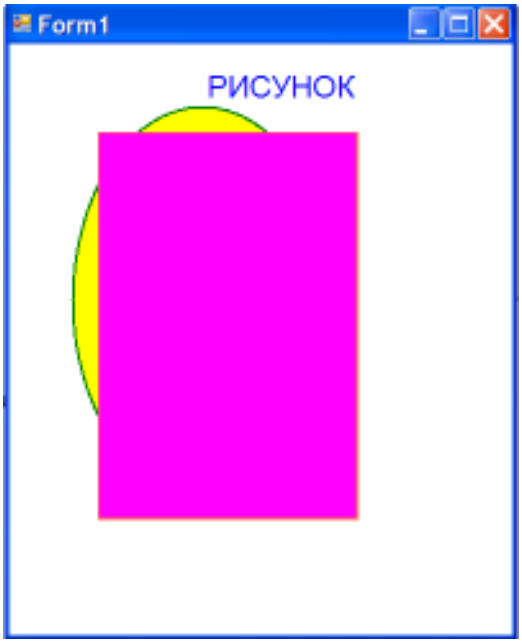

Рисунок 4 – Результат выполнения программы

Упражнение 5

В этом упражнении вы научитесь выводить изображение из файла с использованием GDI+.

1. Создайте проект **Form2_3**.
2. Скопируйте в папку с проектом (Form2_3\Form2_3\bin\Debug) файл

***.bmp**.

3. Отредактируйте код программы в соответствии с приведенным ниже кодом

```
namespace Form2_3
{
   public partial class Form1 : Form
   {
      public Form1()
      {
         InitializeComponent();
      }
      private void Form1_Paint(object sender, PaintEventArgs e)
      {
         DrawImage();
      }
      private void DrawImage()
      {
        Bitmap myBitmap = new Bitmap("*.bmp");

        Graphics g = this.CreateGraphics();

        g.DrawImage(myBitmap, 10, 50);
      }
   }
}
```

4. Запустите программу на выполнение и сравните результат с рисунком 5.
5. Сохраните проект и закройте его.

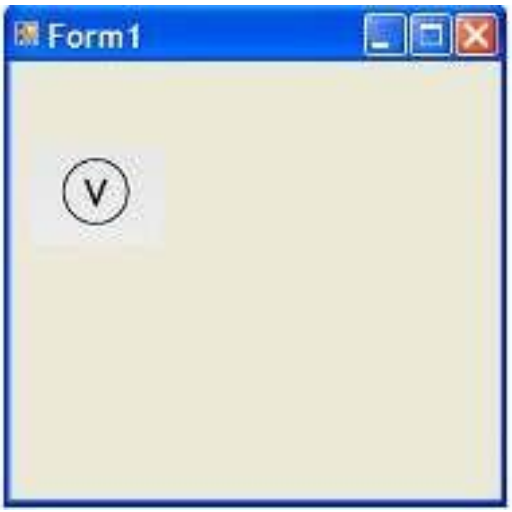

Рисунок 5. – Результат выполнения программы

Упражнение 6

В этом упражнении вы научитесь выводить изображение в заданной точке.

1. Откройте проект **Form2_3**.

2. Добавьте в класс формы объект класса Point для сохранения координат щелчка мыши

```
Point curPoint=new Point(0,0);
```

3. Добавьте обработчик события щелчка по форме

```
private void Form1_MouseClick(object sender, MouseEventArgs e)
        {
          curPoint = e.Location;

          Invalidate();
        }
```

4. Измените метод отрисовки изображения

```
private void DrawImage(Point cp)
        {
          Bitmap myBitmap = new Bitmap("*.bmp");

          Graphics g = this.CreateGraphics();

          g.DrawImage(myBitmap, cp);
        }
```

5. Вызовите DrawImage в методе Form1_Paint

```
private void Form1_Paint(object sender, PaintEventArgs e)
        {
            DrawImage(curPoint);
        }
```

6. Сравните полученный программный код с приведенным ниже:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace Form2_3
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
```

```
        }
        Point curPoint = new Point(0, 0);
        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            DrawImage(curPoint);
        }
        private void DrawImage(Point cp)
        {
            Bitmap myBitmap = new Bitmap("*.bmp");
            Graphics g = this.CreateGraphics();
            g.DrawImage(myBitmap, cp);
        }
        private void Form1_MouseClick(object sender, MouseEventArgs e)
        {
            curPoint = e.Location;
            Invalidate();
        }
    }
}
```

7. Запустите проект на выполнение и убедитесь, что изображение отрисовывается в месте щелчка мышью.
8. Сохраните проект и закройте его.

Одной из наиболее распространенных графических задач является построение графика функции (диаграммы). Библиотека .NET Framework имеет мощный элемент управления Chart для отображения на экране графической информации.

Элемент управления Chart осуществляет всю работу по отображению графиков: строит и размечает оси, рисует координатную сетку, подписывает название осей и самого графика, отображает переданную таблицу в виде всевозможных графиков или диаграмм. В элементе управления Chart можно настроить толщину, стиль и цвет линий, параметры шрифта подписей, шаги разметки координатной сетки и многое другое. В процессе работы программы изменение параметров возможно через обращение к соответствующим свойствам элемента управления Chart. Так, например, свойство AxisX содержит значение максимального предела нижней оси графика и при его изменении во время работы программы автоматически изменяется изображение графика.

Упражнение 7

Разработать приложение Windows Forms, реализующее построение графика изменения напряжения , изменяющегося по закону

$$i \qquad ,$$

где – амплитуда напряжения, ;

– угловая частота, рад/с;

– начальная фаза, град.

$$f,$$

где f – частота, .

График функции построить с помощью элемента управления chart. Ввод параметров построения (константы, начальные и конечные значения, шаг и т.д.) реализовать с использованием элементов TextBox.

1. Создайте проект **Form2_4**.
2. С помощью свойства Name измените имя формы на **MainForm**.
3. Измените размер формы. Для этого, в окне Свойства измените значения свойства Size: ширина – 470, длина – 500.
4. Измените цвет фона формы на белый с помощью свойства BackColor.
5. Откройте панель элементов и разместите на форме элементы управления и укажите их идентификаторы (свойство Name) согласно рисунку 6.

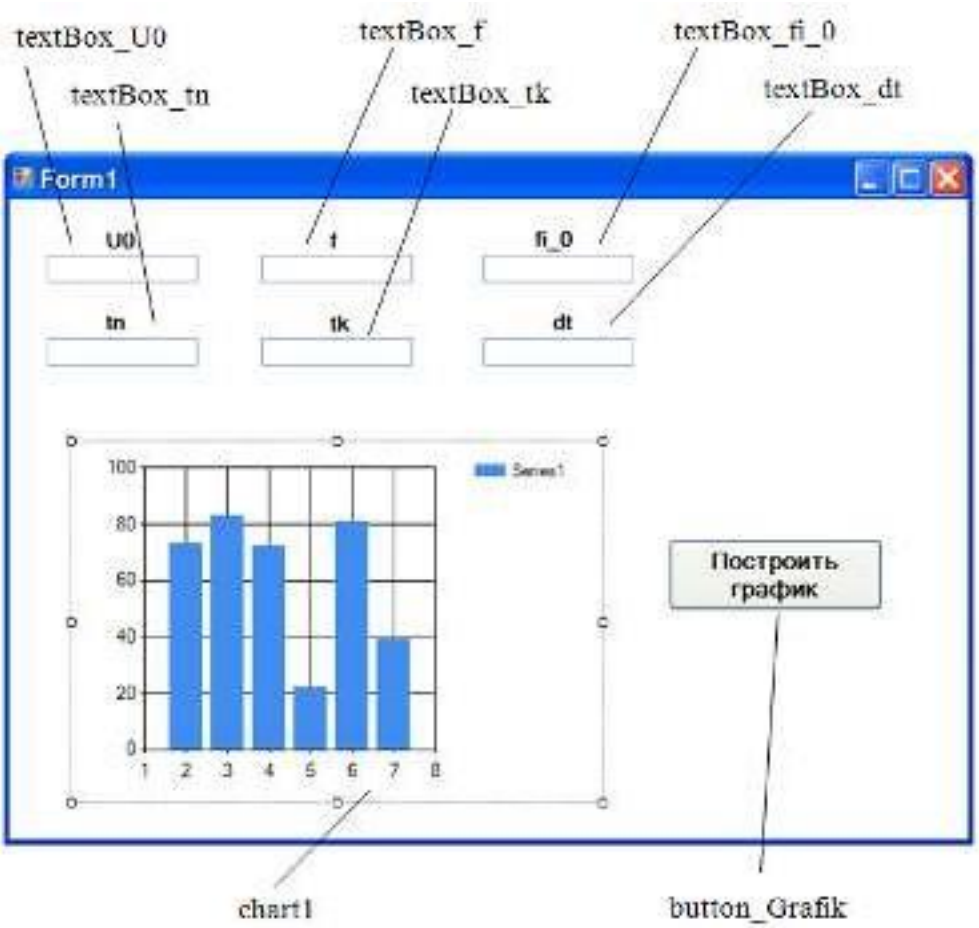

Рисунок 6 – Расположение элементов управления на форме к упражнению 7

6. Для вставки на форму элемента управления chart1 на панели инструментов в списке Данные выберите элемент chart (рисунок 7).

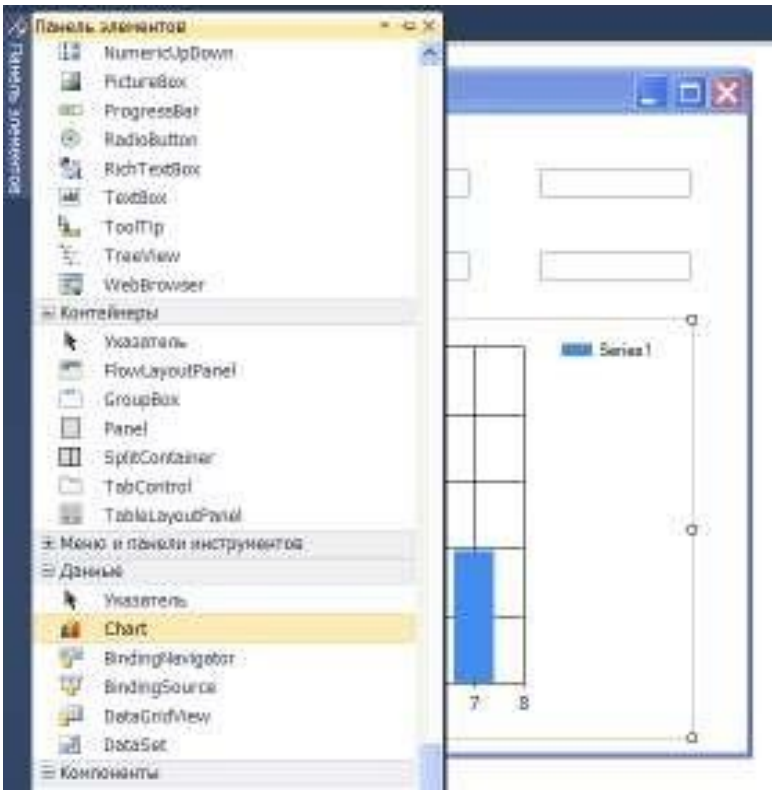

Рисунок 7 – Выбор элемента управления chart

7. Отредактируйте код программы в соответствии с приведенным ниже кодом

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace Form2_4
{
    public partial class MainForm : Form
    {
        public MainForm()
        {
            InitializeComponent();
        }

        private void button_Grafik_Click(object sender, EventArgs e)
        {
          // Получаем значения, введенные пользователем в текстовые поля
          // и преобразовываем их в числовые
          double U0 = Convert.ToDouble(textBox_U0.Text);
```

```
            double f = Convert.ToDouble(textBox_f.Text);
            double fi_0 = Convert.ToDouble(textBox_fi_0.Text);//в градусах
            fi_0 = fi_0 * Math.PI / 180;//в радианах
            double tn = Convert.ToDouble(textBox_tn.Text);
            double tk = Convert.ToDouble(textBox_tk.Text);
            double dt = Convert.ToDouble(textBox_dt.Text);

            // Настраиваем тип графика (команда пишется в одну строку)
            chart1.Series[0].ChartType = System.Windows.Forms.DataVisualization.
            Charting.SeriesChartType.Spline;

            // Очищаем существующие данные
            chart1.Series[0].Points.Clear();

            chart1.ChartAreas[0].AxisX.IsStartedFromZero = true;

            // Добавляем заголовок к графику функции
            chart1.Titles.Add("ГРАФИК ИЗМЕНЕНИЯ НАПРЯЖЕНИЯ");
            chart1.Titles[0].Font = new Font("Utopia", 16);

            // Форматируем цвет фона области диаграммы
            chart1.ChartAreas[0].BackColor = Color.White;


            // Форматировать диаграмму
            chart1.BackColor = Color.Green;
            chart1.BackSecondaryColor = Color.WhiteSmoke;
            chart1.BorderlineColor = Color.DarkGreen;


            for (double t = tn; t <= tk; t += dt)
            {
              double U = U0 * Math.Sin(2 * Math.PI * f * t + fi_0);
              chart1.Series[0].Points.AddXY(t, U);
            }
          }

        }
    }
```

9. Запустите проект на выполнение. Сравните результат выполнения с рисунком 8.
10. Сохраните проект и закройте программу.

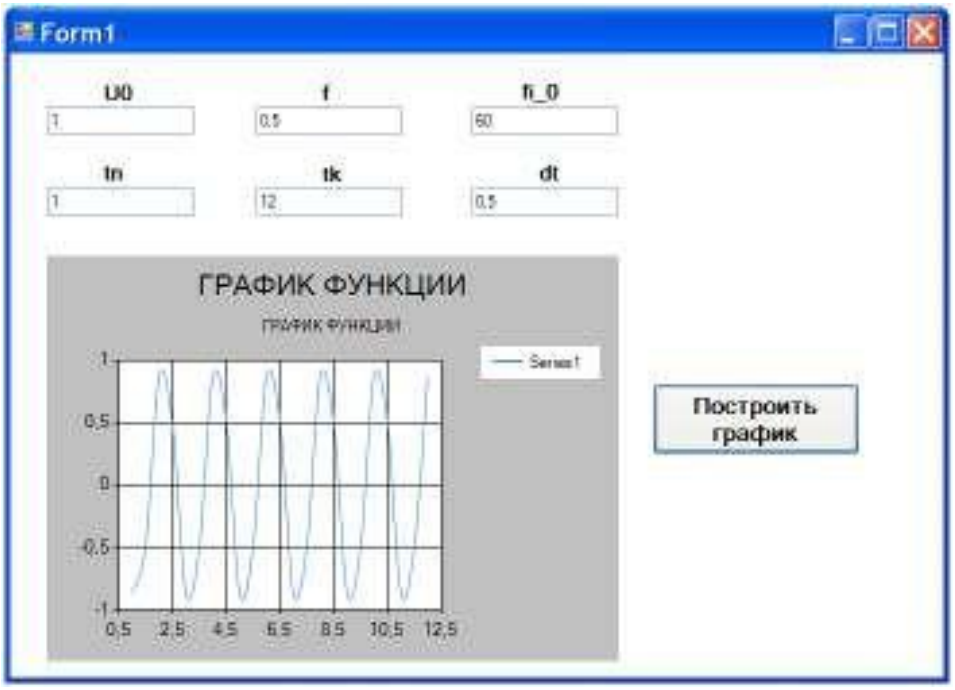

Рисунок 8 – Результат выполнения программы

4 УПРАЖНЕНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

1. Создайте приложение Windows Forms с именем **Form2_1_sr**, реализую- щее возможность отрисовки круга с заливкой цветом в месте щелчка мышью.
2. Создайте приложение Windows Forms с именем **Form2_2_sr**, реализующее вывод графика функции на форму, согласно варианту, указанному преподавателем .

5 СОДЕРЖАНИЕ ОТЧЕТА

Отчет должен содержать листинги программ, результаты выполнения программ и ответы на контрольные вопросы.

6 КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое GDI+?
2. Способы создания графических объектов в GDI+.
3. Как создать ссылку на объект Graphics из объекта PaintEventArgs события

Paint?

4. Как нарисовать графические примитивы в GDI+?
5. Приведите пример кода заливки цветом графического примитива.
6. Приведите пример кода отрисовки объекта на форме в месте щелчка мыши.
7. Приведите пример кода отрисовки текста на форме.